



<NON-REGULAR LANGUAES> <CONVERSION OF NFA TO DFA>

<Theory of Automata and Formal Languages>

<MUHAMMAD ALI SAAGAR> <23021519-095>

<ZAIN ALI YOUSAF> <23021519-122>

OVERVIEW

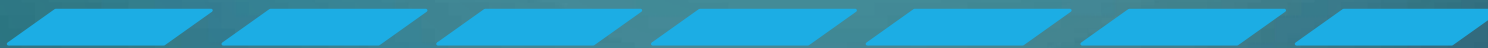
Defining Languages



Conditions for Non-regular Languages



Examples



Pumping Lemma



Procedure



Example



OVERVIEW

Conversion of NFA to DFA



Difference between NFA and DFA



Why to convert NFA to DFA



Steps of converting NFA to DFA



OVERVIEW

Types of NFA and examples :

Type 1 + Examples

Type 2 + Examples

Type 3 + Examples

DEFINING LANGUAGES

- **Definition:** *A language is a set of strings over an alphabet*

Regular languages

Can be recognized
by finite automata

Non-Regular languages

Cannot be
recognized by finite
automata

CONDITIONS FOR NON-REGULARITY OF A LANGUAGE

#1.Dependency

- ⑩ Some parts of a string depend on other parts
- ⑩ Finite Automata can not track these dependencies.

#2.Comparison

- ⑩ Certain languages require comparing numbers of symbols
- ⑩ Finite automata cannot count and compare arbitrarily large numbers

#3.Memorization

- ⑩ Some languages need remembering unlimited information
- ⑩ Finite automata have limited memory and cannot store everything

EXAMPLES OF NON-REGULAR LANGUAGES

$$L = \{a^n b^n : n \geq 0\}$$

$$L = \{a^n b^l c^{n+l} : n, l \geq 0\}$$

$$L = \{g^x h^y j^z : x + y \geq z\}$$

NEED FOR PUMPING LEMMA

How to prove that a language is non-regular?

Pumping Lemma provides a **formal method** for proof.

Statement

- For a regular language L , there exists a **pumping length** p such that:
 - Any string $s \in L$ with $|s| \geq p$ can be split as $s = xyz$
 - Conditions:
 1. $|y| \geq 1$
 2. $|xy| \leq p$
 3. $xy^iz \in L$ for all $i \geq 0$

PUMPING LEMMA

- x = prefix
- y = repeatable section (pumped)
- z = suffix

Language: $L = \{a^n b^n \mid n \geq 0\}$

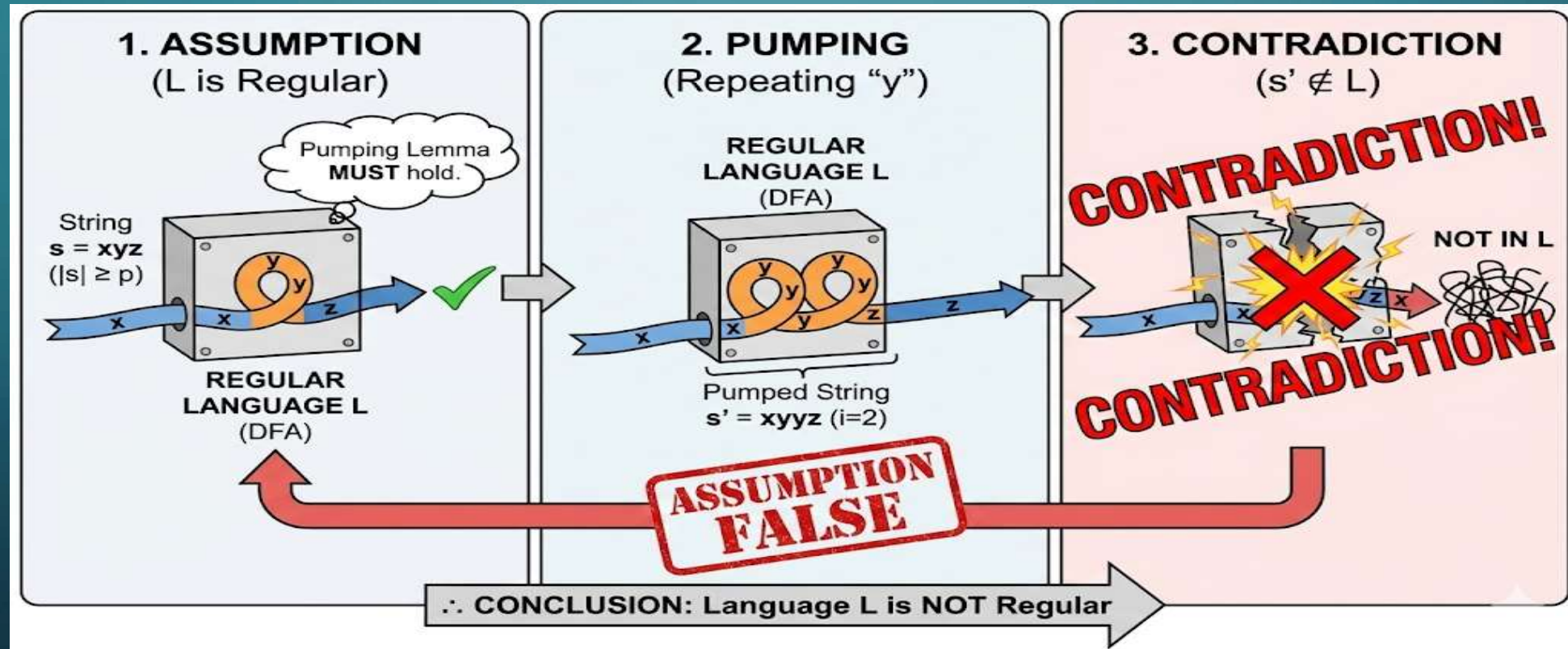
Choose: $w = a^k b^k$

Split: $w = xyz, |xy| \leq k \Rightarrow y$ consists only of a 's

Pump ($i = 2$): $a^{k+|y|} b^k \notin L$

Conclusion: L is non-regular

PUMPING LEMMA



DIFFERENCE BETWEEN NFA AND DFA

Deterministic Finite Automata(DFA)

- ⑩ Exactly one transition for each symbol
- ⑩ ϵ move are not allowed
- ⑩ Need of dead states
- ⑩ Easy to implement in hardware and software

Non-Deterministic Finite Automata(NFA)

- ⑩ Zero , one or more transition for each symbol
- ⑩ ϵ moves are allowed
- ⑩ No need of dead states
- ⑩ Hard to implement in hardware and software

WHY TO CONVERT NFA TO DFA

- ❖ Although NFA is easy to design but DFA is easier to implement .
- ❖ DFA processes input in a single path, so it is faster than NFA.
- ❖ Converted DFA accepts the **same language** as the original NFA.

STEPS FOR CONVERTING NFA TO DFA



Draw ϵ Table if ϵ exists in the NFA



Make NFA table



Make DFA Table from NFA table



Draw DFA structure from DFA table

TYPES OF NFA CONVERSION

❖ Type 1

Only one symbol of same type is passing from one state to other

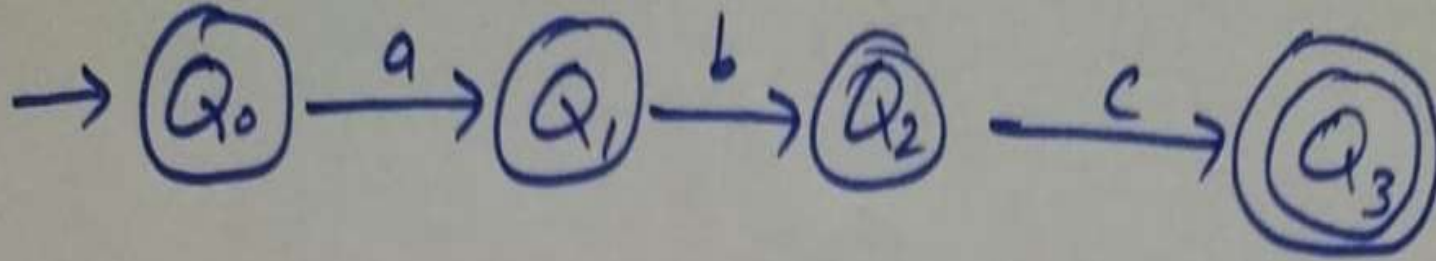
❖ Type 2

More than one symbols of same type are transitioning from one state to other

❖ Type 3

NFA containing ϵ moves

TYPE 1 CONVERSION



NFA - TT

	a	b	c
-Q ₀	Q ₁	∅	∅
Q ₁	∅	Q ₂	∅
Q ₂	∅	∅	Q ₃
+Q ₃	∅	∅	∅

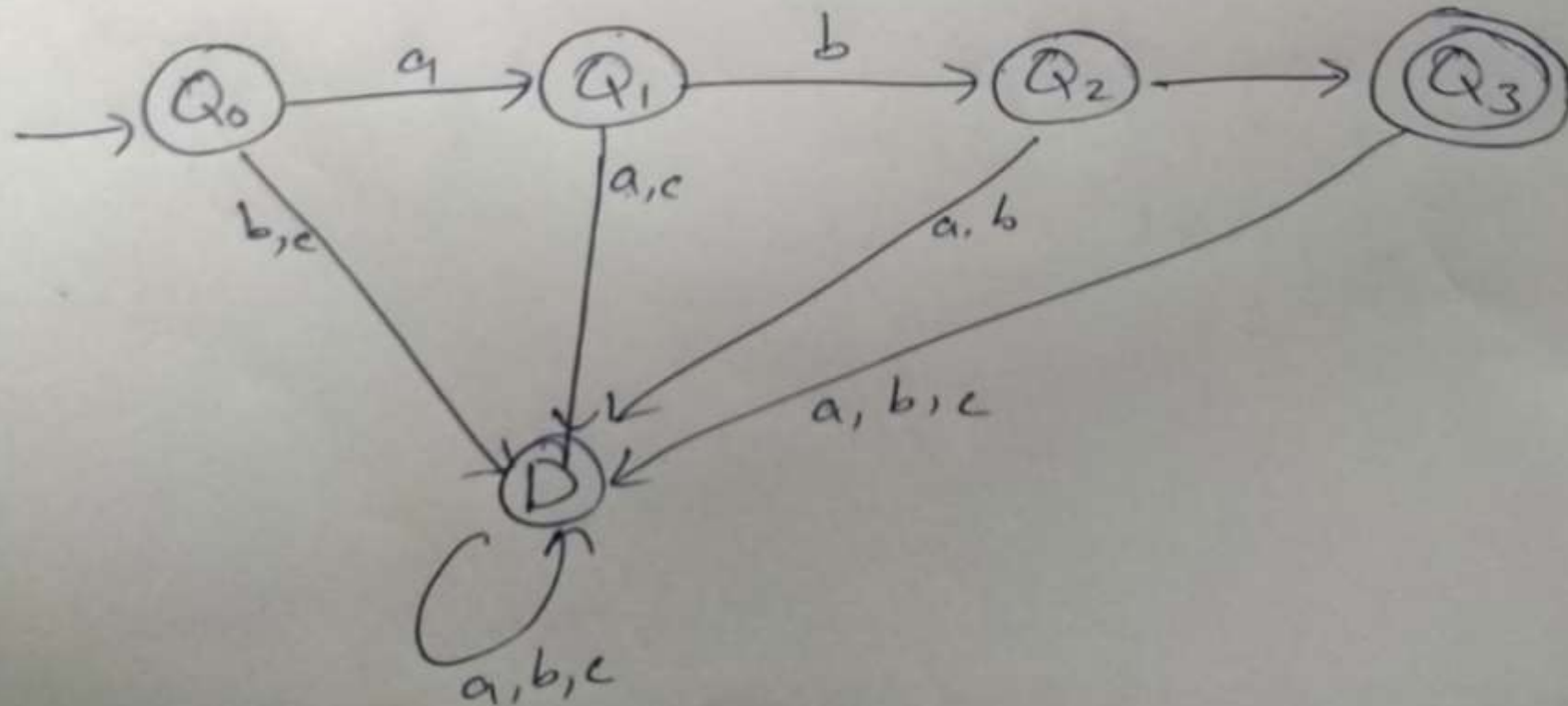


DFA TT

	a	b	c
-Q ₀	Q ₁	D	D
Q ₁	D	Q ₂	D
Q ₂	D	D	Q ₃
+Q ₃	D	D	D
D	D	D	D

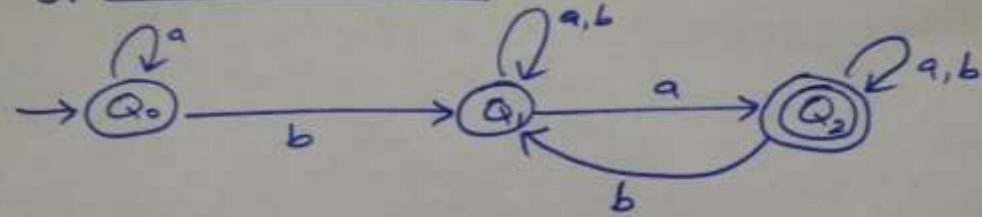
TYPE 1 CONVERSION

DFA



TYPE 2 CONVERSION

Type 2 conversion

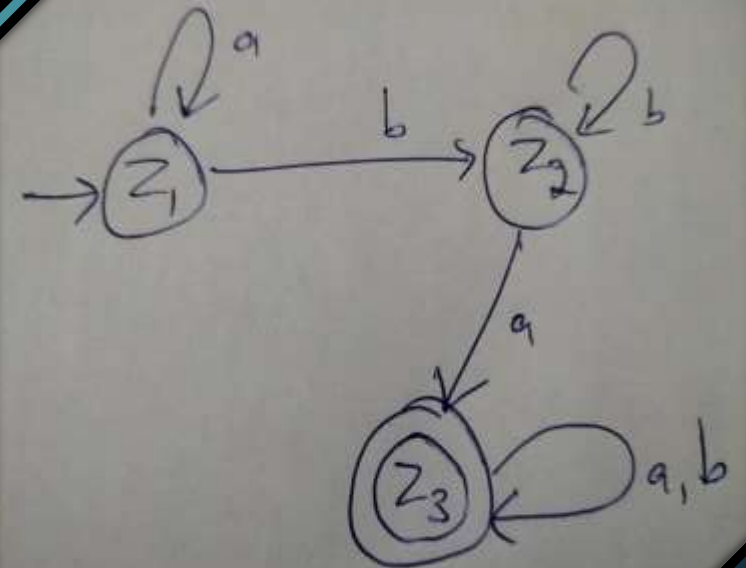


NFA T.T

	a	b
- $z_1 = Q_0$	$z_1 = Q_0$	$z_2 = Q_1$
$z_2 = Q_1$	$z_3 = Q_1, Q_2$	$z_2 = Q_1$
+ $z_3 = Q_1, Q_2$	$z_3 = Q_1, Q_2$	$z_3 = Q_1, Q_2$

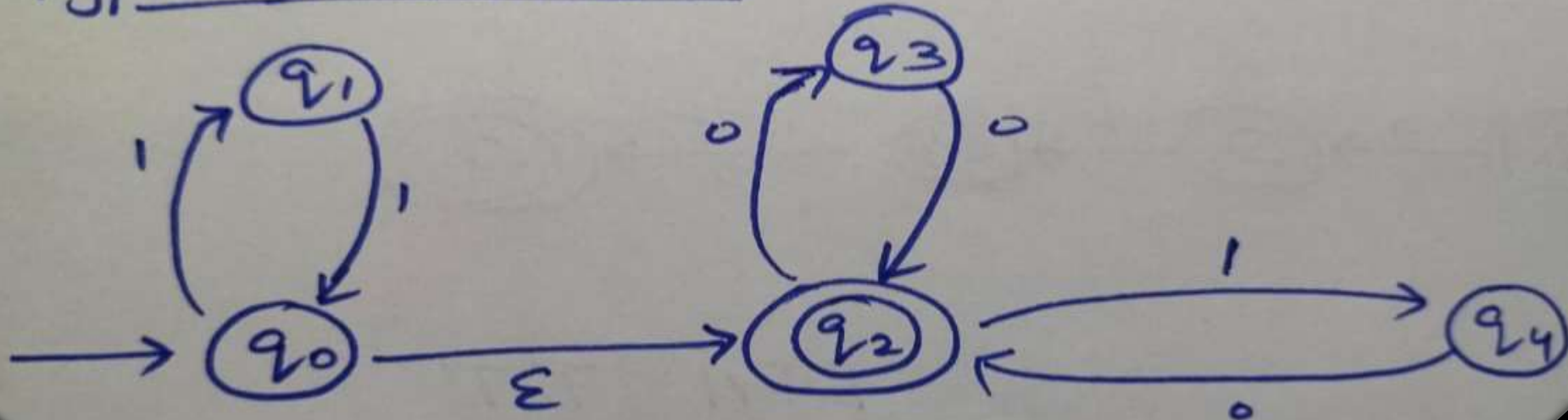
DFA - T.T

	a	b
- z_1	z_1	z_2
z_2	z_3	z_2
+ z_3	z_3	z_3



TYPE 3 CONVERSION

Type 3 Conversion



TYPE 3 CONVERSION

ϵ -Table

States	ϵ
q_0	$q_0 q_2$
q_1	q_1
q_2	q_2
q_3	q_3
q_4	q_4



NFA. T.T

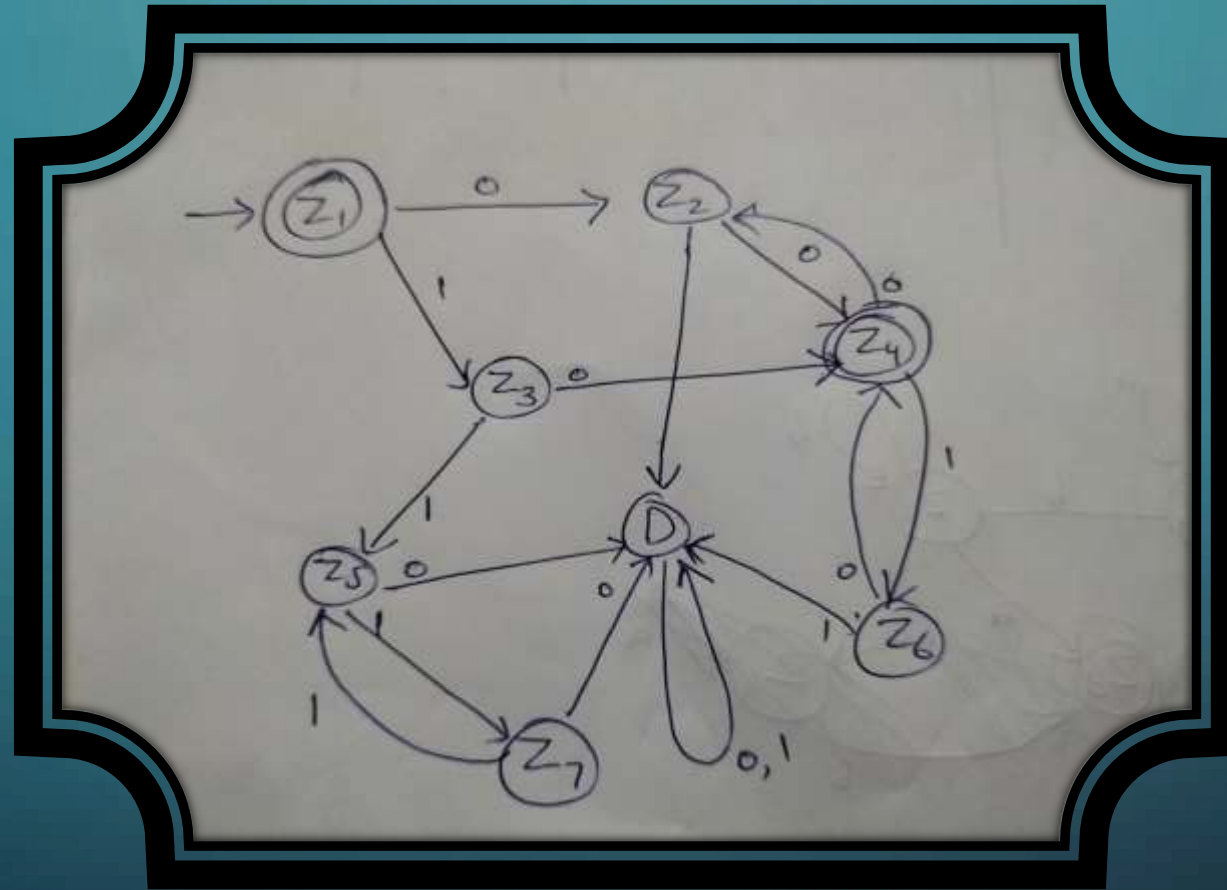
States	0	1
$\pm Z_1 = q_0 q_2$	$Z_2 = q_3$	$Z_3 = q_1 q_4$
$Z_2 = q_3$	$Z_4 = q_2$	ϕ
$Z_3 = q_1 q_4$	$Z_4 = q_2$	$Z_5 = q_0$
$Z_4 = q_2$	$Z_2 = q_3$	$Z_6 = q_4$
$Z_5 = q_0$	ϕ	$Z_7 = q_1$
$Z_6 = q_4$	$Z_4 = q_2$	ϕ
$Z_7 = q_1$	ϕ	$Z_5 = q_0$



DFA TT

States	0	1
$\pm Z_1$	Z_2	Z_3
Z_2	Z_4	D
Z_3	Z_4	Z_5
$+Z_4$	Z_2	Z_6
Z_5	D	Z_7
Z_6	Z_4	D
Z_7	D	Z_5
D	D	D

TYPE 3 CONVERSION



The image features a dark teal background with a subtle gradient. In the corners, there are white line-art illustrations of circuit boards or neural networks, with lines connecting to small circles. The text "Thank You" is centered in a large, bold, white sans-serif font.

Thank You